

车路云协同管控平台 · 部署运行说明

版本：v5.1 · 适用目录：本项目根目录 配套：《全技术栈说明.md》（架构与代码导读）、《实验评估报告.md》（结果）

一、交付内容与运行依赖

组件	说明
后端 backend/	Python 3.10+ (开发用 3.14) + FastAPI + SUMO + PyTorch (CPU 即可)
前端 frontend/	Node 18+ (含 node_modules 时可跳过安装)
路网 networks/	SUMO 路网与车流文件 (base_network 20 信号路口, 及 9 套车流)
模型 backend/models/weights/	MAPPO 权重 (agnostic 跨路网 18 维 + legacy 31 维) 与 ONNX 导出
脚本 backend/scripts/	训练/校准/实验脚本 (可选)
容器化 docker/、docker-compose.yml	Docker 一键部署 (见第三节)

外部依赖 (机器上必须可用) : - SUMO (Windows: 设置 SUMO_HOME 指向安装目录, 如 E:\ ; Linux/mac: sumo 进入 PATH) ——仅在启动仿真时需要; - 可选 Ollama (本地 LLM, ollama pull qwen2.5:1.5b) 或智谱免费 API (.env 配置) 。

二、Windows 直接运行 (推荐开发/演示)

2.1 一键启动

双击根目录 start.bat : 自动检查并安装 Python 依赖 → 起后端 (8000) → 起前端 (5173) → 打开浏览器。

若首次运行依赖安装较慢属正常; 前端已有 node_modules 会跳过。

2.2 手动启动

```
:: 终端 1: 后端
cd backend
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000

:: 终端 2: 前端
cd frontend
npm run dev                :: 访问 http://localhost:5173
```

2.3 LLM 配置

后端启动时读取 `backend/.env`（该文件不入库，需自行准备）：

```
LLM_PROVIDER=zhipu          # 智谱免费 API（推荐）或 ollama
LLM_MODEL=glm-4-flash       # 免费模型：glm-4-flash / glm-4.7-flash
LLM_API_KEY=你的智谱密钥
```

没有 `.env` 时默认走本地 Ollama。也可以在平台右上角智能体栏的"LLM 模型"下拉中随时切换模型（免重启）。

三、Docker 一键部署（换新电脑最快）

3.1 要求

- 安装 Docker Desktop (Windows) 或 docker engine (Linux)，并启动；
- 无需安装 Python / Node / SUMO / Ollama——后端容器内置 SUMO 与全部 Python 依赖。

3.2 一键启动

```
cd 项目根目录
docker compose up -d --build      # 首次构建较慢（下载基础镜像 + pip 装 torch）
# 浏览器打开 http://localhost:5173
docker compose logs -f backend    # 查看后端日志（可选）
docker compose down               # 停止
```

- 后端容器暴露 8000，前端容器 (nginx) 暴露 5173 并把 `/api`、`/ws` 反代到后端；
- 默认 LLM 为本地 Ollama 模式；要智谱请在 `docker-compose.yml` 中按注释配置 `env_file: .env`（或修改 environment）。

3.3 容器内 SUMO 版本说明与备选

后端镜像基于 Debian 软件源的 SUMO（版本可能低于本机 1.27，核心功能不受影响）。若遇到 TraCI 命令兼容性问题，可将 `docker/backend.Dockerfile` 基础镜像替换为 `eclipse/sumo:1.19.0`（官方 SUMO 镜像，含较新 sumo），并在其中补装 `python3/pip` 后按同样步骤安装依赖。

四、环境核对与自检

```
# Python 依赖自检（缺哪个装哪个）
python -c "import fastapi, uvicorn, openai, matplotlib, docx, reportlab; print('deps ok')"
```

```
# SUMO 自检
sumo --version          # Windows 需 SUMO_HOME/bin 在 PATH 或设好 SUMO_HOME
```

```
# 后端健康检查
curl http://127.0.0.1:8000/api/v1/health
```

```
# 模型权重自检（应输出 load: True; False 表示 obs_dim/路径不匹配）
python backend/scripts/verify_model.py
```

五、演示推荐流程（3 分钟内跑通）

1. 启动后选择路网 `base_network` ；
2. 右上角场景选“**区域热点・局部拥堵**”，方案选“方案二（MAPPO/SCOOT）”，点启动——预热约 90 秒后画面出现热点区排队；
3. 观察实时指标（自定义指标卡点“最堵塞道路/路口”可定位到画布）；
4. 对智能体下指令（如“东侧拥堵，请调整最长绿灯并对比前后效果”），观察其调用 `configure_algorithm` 并给出前后对比；
5. 底部“生成报告”导出 md/docx/pdf 评估报告。

六、常见问题

现象	处理
后端报 SUMO 相关错误	检查 SUMO_HOME（Windows）或 sumo 命令（Linux）
启动无车辆	确认已选场景或带车流路网；预热 90 秒内画面空白属正常
智能体报 LLM 失败	检查 .env 密钥；智谱模型可能临时限流，切换 glm-4-flash
端口占用	改 start.bat / docker-compose.yml 端口映射
方案二显示 SCOOT 而非 MAPPO	权重路径/维度不匹配时自动降级；agnostic 权重应与 obs_mode=agnostic 同用
Docker 构建慢	首次拉基础镜像与 torch 较大；构建缓存后增量为秒级

七、目录速览

— start.bat	Windows 一键启动
— docker-compose.yml	容器编排 (backend 8000 + frontend 5173)
— docker/	两个 Dockerfile + nginx 反代配置
— backend/	
— app/	全部后端代码 (api/core/schemes/algorithms/agent/report/eval...)
— models/weights/	MAPPO / STGCN 权重与 ONNX
— scripts/	训练、校准、实验脚本
— requirements.txt	Python 依赖
— frontend/	Vue 3 前端 (vite dev / docker 内 nginx 托管 dist)
— networks/network/	SUMO 路网与车流
— docs/	全技术栈说明 / 部署说明 / 实验评估报告 / 架构文档...